

UTC503

La Programmation Fonctionnelle

Le paradigme fonctionnel ⁽¹⁾

■ Concepts fondamentaux de la Programmation fonctionnelle :

■ **Fonctions** : relation entre deux ensembles A et B tel que tout élément de A soit en relation avec au plus un élément de B. $F(x)$ avec $x \in A$, $F(x)$ est l'image de A et $\in B$.

$$F(x) = x / 2$$

$$F(x1, x2) = x1 + x2$$

■ **Valeurs** : éléments en entrée des fonctions et produits en sortie de celles-ci

■ **Application de fonctions à des valeurs** : production de nouvelles valeurs.

Le paradigme fonctionnel (2)

- **Concepts fondamentaux de la Programmation fonctionnelle :**

- **La programmation fonctionnelle** consiste donc en l'évaluation de combinaisons ou compositions de fonctions mathématiques que l'on appelle expression.

- Fonctions:** $F(x) = x / 2$; $G(x1, x2) = x1 + x2$

- Expression:** $F(G(3, 5))$

- **Les valeurs sont non mutables :** Le changement d'état ou la mutation de données ne peuvent pas être produit par l'évaluation d'une fonction (pas de notion d'affectation)

- « **Pas d'effet de Bord** » : aucune modification de valeurs autres que celles du contexte

Le paradigme fonctionnel (3)

- **Langage fonctionnel :**

- Langage dont la syntaxe et les caractéristiques encouragent la programmation fonctionnelle.

*Quelques langages fonctionnels: **Lisp, Scheme, OCaml***

- **Dans les langages de programmation :** une fonction est l'élément d'un langage qui reçoit un ensemble de paramètres en entrée (la valeur d'entrée), les traite pour produire une valeur qu'elle retourne.

*Les fonctions sont présentes dans la plupart des langages actuels même non fonctionnels: **C, C#, Java etc.***

- **Les fonctions récursives sont au cœur de ce paradigme:**

Exemple: la fonction factorielle $Fact(n) = \text{Si } n=0 \text{ Alors } 1 \text{ Sinon } n * fact(n-1);$

Le paradigme fonctionnel ⁽⁴⁾

■ Propriétés des langages fonctionnels :

- **Pureté** : Un langage fonctionnel est dit pur si il est sans « effet de bord ». Il n'existe aucune instruction d'affectation.

*Les langages fonctionnels purs sont assez rares: **Lisp, Scheme (Common Lisp n'est pas pur)***

- **Transparence référentielle** : « Le résultat d'un programme ne change pas si on remplace une expression par une expression de valeur égale »

Vrai uniquement si les fonctions n'utilisent que leurs valeurs d'entrées (langages fonctionnels purs)

PAS DE CONTEXTE GLOBAL NI DE REFERENCES (facilité de test, pas de problème de concurrence)

Si on a $G(x1, x2) = x1 + x2$ $F(G(1,2))$ est équivalent à $F(3)$

Si on a $H(x) = x + \text{Heure}()$ $H(3) = 17$ (s'il est 14h) $F(H(3))$ et $F(17)$ ne équivalent que s'il est 14h

Le paradigme fonctionnel ⁽⁵⁾

■ Propriétés des langages fonctionnels :

■ **Les fonctions sont des valeurs comme les autres** : durant son évaluation une fonction peut faire appel à d'autres fonctions. Ces invocations de **fonctions** peuvent être **statiques** elles ont été définies dans le corps de la fonction ou **dynamiques** si la fonction est passée en tant que valeur d'entrée (Callback).

Fonctions: $F(x) = x / 2$; $G(x1, x2) = x1 + x2$; $H(x, f) = f(G(x, x))$;

Expression: $H(5, F)$

■ Une fonction peut **retourner une fonction** en tant que résultat

■ Une **fonction qui reçoit une fonction** en paramètre ou **retourne une fonction** est dite : « **Fonction d'ordre Supérieur** »

■ **Langages apparentés à la programmation déclarative** : *SQL, Prolog*

Langages fonctionnels (1)

■ **Typage dans les langages fonctionnels :**

- Les types des arguments passés à une fonction doivent être compatible avec ceux attendus par la fonction
- En cas d'incompatibilité l'évaluation de la fonction ne pourra être effectuée.
- La vérification du type ou typage des arguments est nécessaire

■ **Typage Dynamique :** Le type des arguments est contrôlé au moment de l'évaluation de la fonction

■ **Typage Statique :** Le type des arguments est contrôlé à la compilation ou au moment de l'analyse syntaxique

Langages fonctionnels (2)

■ Typage dans les langages fonctionnels :

- **Typage explicite** : Le type d'un identificateur ou d'une valeur est donné explicitement par l'auteur du code.

Déclaration avec typage explicite: String nom

- **Inférence de type** : Découverte du type d'un identificateur ou d'une valeur en fonction de son utilisation ou de la fonction qui la produit.

Même type que la valeur affectée ou du type résultat:

Expression: nombreLettre(toMajuscule(nom))

- Le type de **nom** devrait être *String* car l'argument de **toMajuscule** doit être un *String* .
- La fonction **toMajuscule** retourne une valeur de type *String* donc l'argument passé à **nombreLettre** est du type *String*

Exemple Programmation Impérative Vs fonctionnelle

■ Calcul de la factorielle d'un nombre:

« La factorielle d'un entier naturel n est le produit des nombres entiers strictement positifs inférieurs ou égaux à n . La factorielle de 0 est 1 »

■ Programmation Impérative :

```
Fact(n) {  
    fact ← 1;  
    Pour i = 1 à n  
        fact ← fact * i;  
    Fin Pour  
    retourner fact;  
}
```

Remarques

- Plusieurs affectations avant d'obtenir le résultat
- La modification de **fact** à un effet de bord
- La factorielle utilise des instructions impératives:
 - Affectations
 - Itération / bouclage

Exemple Programmation Impérative Vs fonctionnelle

- *Programmation fonctionnelle de la factorielle (utilisation de la récursivité) :*

```
Fact(n) {  
    Si n = 0 Alors  
        retourner 1;  
    Sinon  
        retourner n * Fact(n - 1);  
    Fsi  
}
```

Remarques

- Aucune affectation, pas d'effet de bord
- La description de la factorielle est implémentée
- Utilisation de la récursion (!! mémoire et pile !!)
- D'un point de vue mathématique: ressemble à une suite par récurrence $U0, U(n) = U(n-1) * n$

Comparaison Programmation Impérative Vs fonctionnelle

	Prog. impérative	Prog. Fonctionnelle
Approche du programmeur	- Algorithme - Gestion modification des états	- Informations en entrées - Transformations requises
Modification de l'état	IMPORTANTE	AUCUNE
Ordre d'exécution	IMPORTANT	PEU IMPORTANT
Contrôle du flux principal	Boucles, Conditions, Appels de Fonctions et de Méthodes	Peu important (respect priorités des fonctions)
Unité de manipulation principale	Instances de modules ou de classes	Fonctions

Conclusion

- **La programmation impérative :**

- Impose une séquence d'instruction dont l'ordre est important.
- Est basée sur la mutabilité des variables.

- **La programmation fonctionnelle :**

- Remet en cause notre façon de résoudre un problème.
- Peut s'utiliser même dans des langages fortement impératifs.
- Promeut la composabilité (assemblage de fonctions)
- Proscrit les effets de bord et les risques de bugs induits.

En fonctionnel on exprime dans le langage ce qu'est une fonction et on laisse décider la machine comment la réaliser.

Programmation Fonctionnelle

EXERCICES

Calcul d'un nombre à la puissance

■ *Implémentation du calcul d'un **nombre** élevé à une puissance **n***

puissance (nombre : Entier, n : Entier) : Entier;

Exemple : puissance(2, 3) = 2 x 2 x 2 = 8

Proposer 2 versions de ce calcul : l'une en programmation impérative, l'autre en programmation fonctionnelle en utilisant la récursivité.

Calcul d'un terme de la suite de Fibonacci

■ *Implémentation du calcul d'un terme de la suite de Fibonacci*

fib(nombre : Entier) : Entier;

La suite de Fibonacci est une suite d'entiers dans laquelle chaque terme est la somme des deux termes qui le précèdent. Fib(n) est définie de la manière suivante :

Fib(0) = 0, Fib(1) = 1, Fib(n) = Fib(n-1) + Fib(n-2) (pour $n \geq 2$)

Exemple : fib(2) = 0 + 1 = 1; fib(3) = 1 + 1 = 2 etc.

Proposer 2 versions de ce calcul : l'une en programmation impérative, l'autre en programmation fonctionnelle en utilisant la récursivité.

Récurtivité

■ Implémentation d'une fonction récursive :

1 - Déterminer le cas terminal

exemples : n Puissance 0 $\rightarrow$ 1

ou Factorielle de 0 $\rightarrow$ 1

2 - Exprimer le ou les cas de récurions

*exemples : n Puissance $p \rightarrow n * n$ Puissance $p - 1$*

*ou Factorielle de $n \rightarrow n * \text{Factorielle de } n - 1$*

■ **Réursion vs Itération:**

<http://www.science-du-numerique.fr/difference-entre-recursion-et-iteration>

Réursion : complexité exponentielle

Itération : complexité linéaire

Il faut privilégier l'itératif, sauf si nombre de récurions faible, car le récursif restera plus lisible

Liste « fonctionnelle non mutable » (1)

■ Nous allons utiliser une liste non mutable. C'est une Liste qu'il est impossible de modifier une fois créée. Par conséquent les ajouts et retraits d'élément sont interdits.

■ Cette liste aura 3 méthodes de niveau instance :

isEmpty() : retourne **true** si la liste est vide

getHead() : retourne le premier élément de la liste

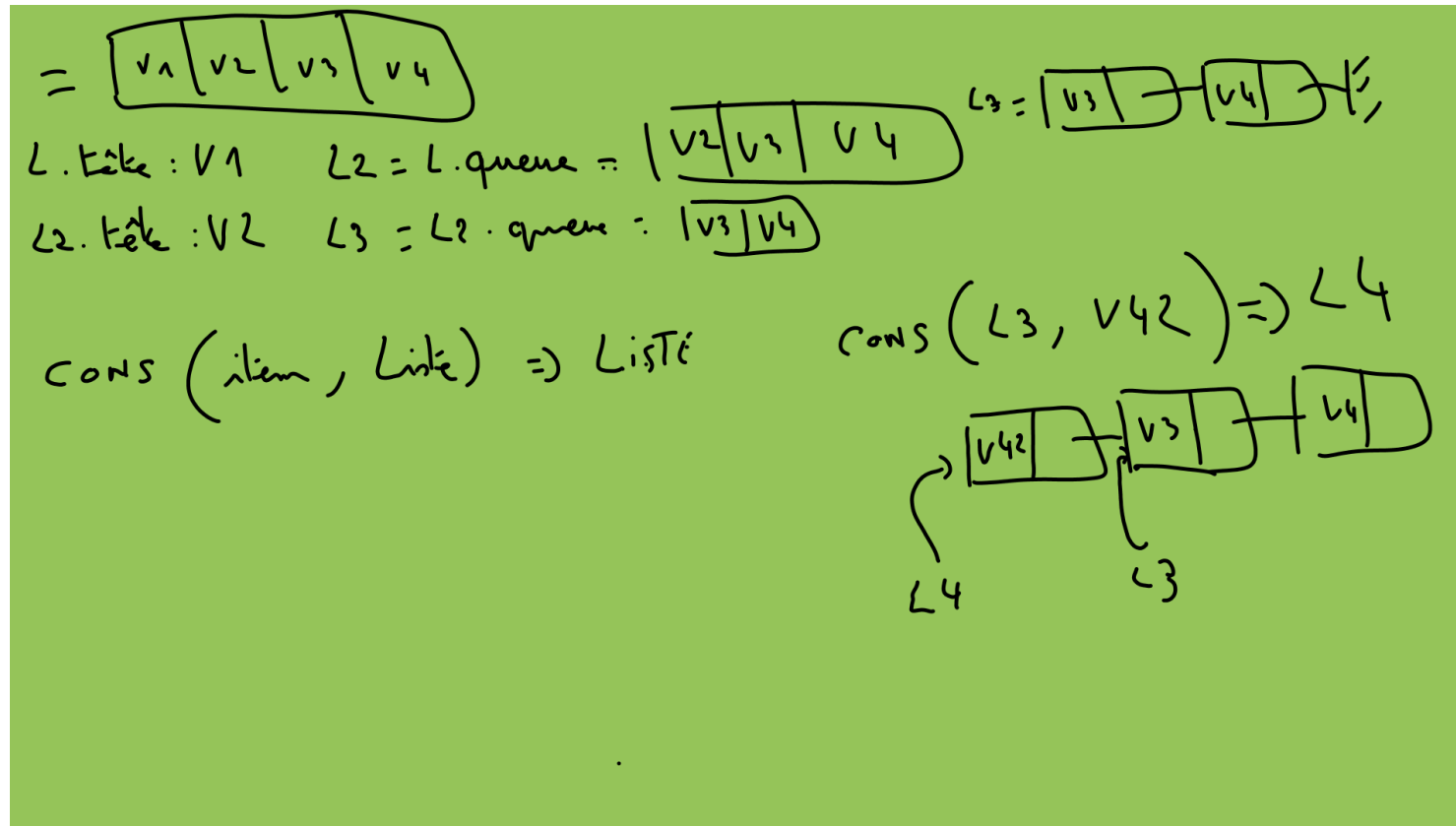
getTail() : retourne la liste privée de son premier élément

■ La construction de cette liste se fera par les méthodes de niveau classe :

public static <T> Liste<T> of(T... items) : qui crée et retourne une liste avec les éléments items

public static <T> Liste<T> cons(T item, Liste<T> liste) : qui crée et retourne une liste comportant item en tant que *head* et liste en tant que *tail*

Liste « fonctionnelle non mutable » (1)



Liste « fonctionnelle non mutable » (2)

■ Utilisation de la liste :

Classe Liste : https://amio-fit.2isa.eu/pluginfile.php/31628/mod_resource/content/0/Liste.java

```
Liste<Integer> liste1 = Liste.of(1,2,3);  
System.out.println(liste1);           // affiche : ( 1 2 3)
```

```
Liste<Integer> liste2 = Liste.cons(5, liste1);  
System.out.println(liste2);           // affiche : (5 1 2 3)
```

```
System.out.println(liste2.getHead()); // affiche : 5  
System.out.println(liste2.getTail()); // affiche : (1 2 3)
```

Taille d'une liste

■ *Implémentation du calcul de la taille d'une liste*

```
public static int tailleListe(Liste<?> liste)
```

Exemple : `tailleListe(Liste.of(1, 2, 4, 6)) = 4`

Proposer 2 versions de ce calcul : l'une en programmation impérative, l'autre en programmation fonctionnelle en utilisant la récursivité.

Elimination éléments d'une liste

- *Implémentation d'une fonction pour éliminer les occurrences d'un élément dans une liste*

```
public static <T> Liste<T> elimine(Liste<T> liste, T aEliminer)
```

Exemple : $\text{elimine}(\text{Liste.of}(1, 2, 4, 6), 2) = (1, 4, 6)$

Proposer une implémentation en programmation fonctionnelle en utilisant la récursivité.

Somme des éléments d'une liste

■ *Implémentation d'une fonction pour retourner la somme des éléments d'une liste (réduction)*

```
// Version sur liste d'entier  
public static Integer somme(Liste<Integer> liste)
```

Exemple : somme(Liste.of(1, 2, 4, 6)) = 13

Proposer une implémentation en programmation fonctionnelle en utilisant la récursivité.

Est-il possible de rendre plus générique cette fonction en lui permettant de sommer des listes quel que soit le type d'élément ?

Les Interfaces Fonctionnelles en Java

- *Interface ne proposant qu'une seule fonction*
- Annotée **@FunctionalInterface** (mais pas obligatoire)
- Permet son implémentations en tant que **classe** mais aussi en tant que **Lambda Expression**
- Les Interfaces pour la programmation fonctionnelles sont dans le package **java.util.function** <https://docs.oracle.com/javase/8/docs/api/java/util/function/package-summary.html>
- Elles permettent l'utilisation de « **Fonctions d'ordre Supérieur** »

Le package java.util.function ⁽¹⁾

■ Comporte un ensemble **d'interfaces fonctionnelles génériques**

Interface fonctionnelle : Interface avec une seule méthode (abstraite)

```
BiConsumer<T,U>, BiFunction<T,U,R>, BinaryOperator<T>, BiPredicate<T,U>,  
BooleanSupplier, Consumer<T>, DoubleBinaryOperator, DoubleConsumer,  
DoubleFunction<R>, DoublePredicate, DoubleSupplier, DoubleToIntFunction,  
DoubleToLongFunction, DoubleUnaryOperator, Function<T,R>, IntBinaryOperator,  
IntConsumer, IntFunction<R>, IntPredicate, IntSupplier, IntToDoubleFunction,  
IntToLongFunction, IntUnaryOperator, LongBinaryOperator, LongConsumer,  
LongFunction<R>, LongPredicate, LongSupplier, LongToDoubleFunction,  
LongToIntFunction, LongUnaryOperator, ObjDoubleConsumer<T>, ObjIntConsumer<T>,  
ObjLongConsumer<T>, Predicate<T>, Supplier<T>, ToDoubleBiFunction<T,U>,  
ToDoubleFunction<T>, ToIntBiFunction<T,U>, ToIntFunction<T>, ToLongBiFunction<T,U>,  
ToLongFunction<T>, UnaryOperator<T>
```

Le package `java.util.function` (2)

(vue partielle)

Interface	Description
<code>BiConsumer<T,U></code>	Represents an operation that accepts two input arguments and returns no result.
<code>BiFunction<T,U,R></code>	Represents a function that accepts two arguments and produces a result.
<code>BinaryOperator<T></code>	Represents an operation upon two operands of the same type, producing a result of the same type as the operands.
<code>BiPredicate<T,U></code>	Represents a predicate (boolean-valued function) of two arguments.
<code>BooleanSupplier</code>	Represents a supplier of <code>boolean</code> -valued results.
<code>Consumer<T></code>	Represents an operation that accepts a single input argument and returns no result.
<code>DoubleBinaryOperator</code>	Represents an operation upon two <code>double</code> -valued operands and producing a <code>double</code> -valued result.
<code>DoubleConsumer</code>	Represents an operation that accepts a single <code>double</code> -valued argument and returns no result.
<code>DoubleFunction<R></code>	Represents a function that accepts a double-valued argument and produces a result.
<code>DoublePredicate</code>	Represents a predicate (boolean-valued function) of one <code>double</code> -valued argument.
<code>DoubleSupplier</code>	Represents a supplier of <code>double</code> -valued results.
<code>DoubleToIntFunction</code>	Represents a function that accepts a double-valued argument and produces an <code>int</code> -valued result.
<code>DoubleToLongFunction</code>	Represents a function that accepts a double-valued argument and produces a long-valued result.
<code>DoubleUnaryOperator</code>	Represents an operation on a single <code>double</code> -valued operand that produces a <code>double</code> -valued result.
<code>Function<T,R></code>	Represents a function that accepts one argument and produces a result.

Utilisation des interfaces fonctionnelles (1)

- Création d'une classe qui implémente l'interface :

```
/**
 * Classe qui implémente l'interface
 */
public class LaFonctionParClasse implements Function<String, Integer> {
    @Override
    public Integer apply(String t) {
        return t.replace("a", "").length();
    }
}

// Utilisation de la classe implémentant l'interface
Function<String, Integer> fonctionParClasse= new LaFonctionParClasse();
System.out.println(fonctionParClasse.apply("babar"));
```

Utilisation des interfaces fonctionnelles (2)

- Création d'une classe anonyme qui implémente l'interface :

```
// Classe anonyme
Function<String, Integer> fonctionParClasseAnonyme = new Function<String, Integer>() {
    @Override
    public Integer apply(String t) {
        return t.replace("a", "").length();
    }
};
```

```
// Utilisation de la classe anonyme implémentant l'interface
System.out.println(fonctionParClasseAnonyme.apply("babar"));
```

Utilisation des interfaces fonctionnelles (3)

- Création à partir d'une lambda expression :

```
//Lambda
Function<String, Integer> fonctionParLambda= (t) -> t.replace("a", "").length();

// Utilisation de la fonction créée par la lambda expression
System.out.println(fonctionParLambda.apply("babar"));
```

Filtrer les éléments d'une liste

- *Implémentation d'une fonction pour filtrer les éléments d'une liste d'après une condition*

```
public static <T> Liste<T> filtrer(Liste<T> liste, Predicate<T> fonctionFiltre)
```

Proposer une implémentation en programmation fonctionnelle en utilisant la récursivité.

Utiliser votre fonction filtrer pour ne conserver que les nombres pairs d'une liste d'entiers.

Création dynamique d'une fonction

- *Implémentation d'une fonction pour créer un multiplicateur par le nombre passé en paramètre.*

```
// Version sur liste d'entier  
public static Function<Integer, Integer> creerMultiplicateur(Integer multPar)
```

Proposer une implémentation de cette fonction.